

1. Variáveis e Objetos

O que são?

- **Variáveis** armazenam dados na memória do computador para uso posterior.
- Devem ter nomes que sigam estas regras:
 - Não começar com números.
 - Não usar caracteres especiais como #, \$, %, &.
 - Para nomes compostos, use **underscore** (_) para separar palavras:
`nome_completo`.

2. Tipos de Dados

Primitivos:

- `int`: Números inteiros. Ex.: `10`
- `float`: Números decimais. Ex.: `3.14`
- `str`: Texto. Ex.: `"Olá"`
- `bool`: Verdadeiro (`True`) ou Falso (`False`).

```
numero = int(input("Digite um número inteiro: ")) # Converte texto para número inteiro
print("O número digitado foi:", numero)
```

3. Operadores

- **Aritméticos:**
 - `+`, `-`, `*`, `/`, `**` (potência), `//` (divisão inteira), `%` (resto).
- **Comparação:**
 - `==`, `!=`, `>`, `<`, `>=`, `<=`.

```
x = 10
y = 3
print("Soma:", x + y)
print("Maior que 5?", x > 5)
```

- **Operadores lógicos:**
 - **and**: Retorna `True` se todas as condições forem verdadeiras.
 - **or**: Retorna `True` se pelo menos uma condição for verdadeira.
 - **not**: Inverte o valor lógico

```
1 idade = 20
2 altura = 1.60
3 if idade > 18 and altura > 1.50:
4     print("Você pode participar!")
5 else:
6     print("Requisitos não atendidos.")
7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORT

PS C:\Users\maria\Desktop\FCUL\1 ano\programação
Você pode participar!

4. Módulos

São bibliotecas prontas para usar.

- **Math**: Operações matemáticas avançadas

```
4 from math import sqrt, pow
5 print("Raiz quadrada de 16:", sqrt(16))
6 print("2 elevado a 3:", pow(2, 3))
7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\maria\Desktop\FCUL\1 ano\programação I\RESUMO> &
Raiz quadrada de 16: 4.0
2 elevado a 3: 8.0

- **Random:** Geração de números aleatórios.

```
1 from random import randint
2 print("Número aleatório:", randint(1, 10))
3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\martha\Desktop\FCUL\1 ano\programação I\RESUMO>
Número aleatório: 7

- **Datetime:** Trabalha com datas e horários.

```
1 from datetime import date
2 print("Ano atual:", date.today().year)
3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\martha\Desktop\FCUL\1 ano\programação I\RESUMO> & C:/l
Ano atual: 2024

5. Estruturas Condicionais

- **if** (se), **else** (se não), **elif** (se não se).

```
1 idade = int(input("Digite sua idade: "))
2 if idade < 18:
3     print("Você é menor de idade.")
4 elif idade < 60:
5     print("Você é adulto.")
6 else:
7     print("Você é idoso.")
8
9
10
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\martha\Desktop\FCUL\1 ano\programação I\RES
Digite sua idade: 10
Você é menor de idade.

6. Estruturas de Repetição

Uso do **break** e **continue**

- **break:** Sai imediatamente do loop.
- **continue:** Pula para a próxima iteração.
- **for:** Repetição com contador. Ideal para quando sabemos quantas vezes queremos repetir algo.

```
1 for i in range(1, 6):
2     print(f"Contando: {i}")
3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\martha\Desktop\FCUL\1 ano\programação I\RESU
Contando: 1
Contando: 2
Contando: 3
Contando: 4
Contando: 5

- **while:** Repetição enquanto uma condição for verdadeira. Útil para repetições com condições dinâmicas.

```
1 contador = 0
2 while contador < 5:
3     print("Repetindo...")
4     contador += 1
5
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\martha\Desktop\FCUL\1 ano\programaç
Repetindo...
Repetindo...
Repetindo...
Repetindo...
Repetindo...

7. Listas

Conjunto de elementos mutáveis.

O que é uma lista?

- Estrutura que armazena múltiplos valores.
- Aceita diferentes tipos de dados.

```
1 lista = ["maçã", "banana", 42, True]
```

• Como usar?

- Criar: `lista = [1, 2, 3]`
- Adicionar: `lista.append(laranja)`
- Inserir em uma posição específica: `lista.insert(1, "uva")`
- Remover: `lista.pop()` ou `lista.remove(valor)`
- Ordenar: `lista.sort()`

```
1 frutas = ["maçã", "banana", "laranja"]
2 print("Lista atualizada:", frutas)
3
4 frutas.append("uva")
5 print("Lista atualizada:", frutas)
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\marta\Desktop\FCUL\1 ano\programação I\RESUM
Lista atualizada: ['maçã', 'banana', 'laranja']
Lista atualizada: ['maçã', 'banana', 'laranja', 'uva']
```

• Concatenar Listas:

```
1 lista1 = [1, 2]
2 lista2 = [3, 4]
3 lista3 = lista1 + lista2
4 print(lista3) # [1, 2, 3, 4]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\marta\Desktop\FCUL\1 ano\programação I\
[1, 2, 3, 4]
```

8. Dicionários

Estruturas de dados que armazenam valores em pares `chave: valor`.

• Exemplo:

```
1 pessoa = {"nome": "Ana", "idade": 25, "cidade": "São Paulo"}
2 print("Nome:", pessoa["nome"])
3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\marta\Desktop\FCUL\1 ano\programação I\RESUMO> & C:/Users/m
Nome: Ana
```

9. Funções

Permitem criar blocos reutilizáveis de código.

• Como criar?

```
1 def saudacao(nome):
2     return f"Olá, {nome}!"
3
4 print(saudacao("Maria"))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\marta\Desktop\FCUL\1 ano\programação I
Olá, Maria!
```

- **Parâmetros Opcionais:**

```
1 def soma(a, b=0):
2     return a + b
3
4 print(soma(5))
5 print(soma(5, 3))
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\marta\Desktop\FCUL\1 ano\programação I\RESUMO>
5
8

- **Documentação com Docstrings**

Adicione descrições para funções:

```
1 def soma(a, b):
2     """Retorna a soma de dois números."""
3     return a + b
4
5 print(soma.__doc__) # "Retorna a soma de dois números."
6
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\marta\Desktop\FCUL\1 ano\programação I\RESUMO> & C:/Use
Retorna a soma de dois números.

9. Tuplas

Semelhantes a listas, mas **imutáveis**.

```
1 tupla = (1, 2, 3)
2 print(tupla[0]) # Acessando o primeiro elemento
3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\marta\Desktop\FCUL\1 ano\programação I\RESUMO>
1

Principais métodos

- `len(tupla)`: Tamanho.
- `sorted(tupla)`: Retorna uma lista ordenada.

```

11.py > ...
1  # Variáveis e Tipos de Dados
2  tarefas = [] # Lista para armazenar as tarefas
3  usuario = {"nome": "João", "idade": 25} # Dicionário para o usuário
4
5  # Funções
6  def mostrar_menu():
7      """Exibe o menu principal."""
8      print("\nMenu:")
9      print("1. Adicionar Tarefa")
10     print("2. Listar Tarefas")
11     print("3. Remover Tarefa")
12     print("4. Sair")
13
14     def adicionar_tarefa(tarefa):
15         """Adiciona uma nova tarefa à lista."""
16         tarefas.append(tarefa) #.append() adiciona a tarefa na lista
17         print(f"Tarefa '{tarefa}' adicionada!")
18
19     def listar_tarefas():
20         """Lista todas as tarefas."""
21         if tarefas:
22             print("\nSuas tarefas:")
23             for i, tarefa in enumerate(tarefas):
24                 print(f"{i + 1}. {tarefa}")
25         else:
26             print("\Nenhuma tarefa cadastrada.")
27
28     def remover_tarefa(indice):
29         """Remove uma tarefa pelo índice."""
30         try:
31             tarefa_removida = tarefas.pop(indice - 1)
32             print(f"Tarefa '{tarefa_removida}' removida!")
33         except IndexError:
34             print("Índice inválido. Tente novamente.")
35
36     # Programa Principal
37     print(f"Bem-vindo, {usuario['nome']}! Vamos gerenciar suas tarefas.")
38
39     while True: # Estrutura de Repetição
40         mostrar_menu()
41         opcao = input("Escolha uma opção (1-4): ")
42
43         # Estruturas Condicionais
44         if opcao == "1":
45             nova_tarefa = input("Digite a nova tarefa: ")
46             adicionar_tarefa(nova_tarefa)
47         elif opcao == "2":
48             listar_tarefas()
49         elif opcao == "3":
50             listar_tarefas()
51             indice = int(input("Digite o número da tarefa para remover: "))
52             remover_tarefa(indice)
53         elif opcao == "4":
54             print("Saindo do programa. Até logo!")
55             break # Interrompe o loop
56         else:
57             print("Opção inválida. Tente novamente.")

```

Azul:

Explicação:

- Essa função apenas **exibe o menu** com as opções disponíveis.
- Usamos `\n` no `print` para pular uma linha e melhorar a apresentação.
- O menu será mostrado repetidamente sempre que o programa voltar ao início do loop principal.

Rosa:

Explicação:

- O argumento `tarefa` é o que o usuário digitou (uma string representando a nova tarefa).
- Usamos o método `append()` para adicionar a tarefa à lista `tarefas`.
- Uma mensagem de confirmação é exibida ao usuário.

Exemplo de Execução:

Entrada: "Estudar Python"

Saída: Tarefa 'Estudar Python' adicionada!

Amarelo:

Explicação:

- **Se `tarefas` estiver vazia:** Exibe "Nenhuma tarefa cadastrada."
- **Caso contrário:**
 - Percorre a lista `tarefas` com `for` e `enumerate()`.
 - O `enumerate()` retorna o índice (`i`) e o valor (`tarefa`).
 - Os índices começam em `0`, mas adicionamos `1` para mostrar números mais intuitivos ao usuário.

Vermelho:

Explicação:

1. `tarefas.pop(indice - 1):`
 - Remove a tarefa pelo índice ajustado (o usuário vê números começando de 1, mas listas começam de 0).
 - Armazena a tarefa removida em `tarefa_removida` para mostrar ao usuário.
2. **`try` e `except`:**
 - Caso o usuário forneça um índice inválido (ex.: número fora da lista), ocorre um erro. Com o `try/except`, exibimos uma mensagem amigável ao invés de quebrar o programa.

Exemplo de Execução:

Entrada: Remover tarefa 2 (tarefa correspondente: "Fazer compras")

Saída: Tarefa 'Fazer compras' removida!

Roxo:

Saudação inicial:

```
print(f"Bem-vindo, {usuario['nome']}! Vamos gerenciar suas tarefas.")
```

Loop Principal:

```
while True:
```

- O programa continua executando enquanto o usuário não escolher a opção de sair (4).

Mostrar o menu:

```
mostrar_menu()
```

- A cada iteração, chamamos a função `mostrar_menu()`.

Receber entrada do usuário:

```
opcao = input("Escolha uma opção (1-4): ")
```

- O usuário digita a opção desejada.

Estruturas condicionais:

- **Opção 1:** Adicionar tarefa.
- **Opção 2:** Listar tarefas.
- **Opção 3:** Remover tarefa.
- **Opção 4:** Encerrar o programa com `break`.

Mensagem de erro para entrada inválida:

```
else:
```

```
    print("Opção inválida. Tente novamente.")
```

- Se o usuário digitar algo fora das opções 1 a 4, mostramos uma mensagem de erro.